



北京航空航天大学
BEIHANG UNIVERSITY



数据结构与程序设计（信息类）

Data Structure & Programming

北京航空航天大学 数据结构课程组

软件学院 谭火彬

2024年春



提纲：排序

- ◆ 7.1 排序基础
- ◆ 7.2 插入排序
- ◆ 7.3 选择排序
- ◆ 7.4 冒泡排序
- ◆ 7.5 谢尔排序
- ◆ 7.6 堆排序
- ◆ 7.7 二路归并排序
- ◆ 7.8 快速排序



Sort Benchmark Home Page

Top Results

	Daytona	Indy
Gray	2016, 44.8 TB/min Tencent Sort 100 TB in 134 Seconds 512 nodes x (2 OpenPOWER 10-core POWER8 2.926 GHz, 512 GB memory, 4x Huawei ES3600P V3 1.2TB NVMe SSD, 100Gb Mellanox ConnectX4-EN) Jie Jiang, Lixiong Zheng, Junfeng Pu, Xiong Cheng, Chongqing Zhao Tencent Corporation Mark R. Nutter, Jeremy D. Schaub	2016, 60.7 TB/min Tencent Sort 100 TB in 98.8 Seconds 512 nodes x (2 OpenPOWER 10-core POWER8 2.926 GHz, 512 GB memory, 4x Huawei ES3600P V3 1.2TB NVMe SSD, 100Gb Mellanox ConnectX4-EN) Jie Jiang, Lixiong Zheng, Junfeng Pu, Xiong Cheng, Chongqing Zhao Tencent Corporation Mark R. Nutter, Jeremy D. Schaub
Cloud	2016, \$1.44 / TB NADSort 100 TB for \$144 394 Alibaba Cloud ECS ecs.n1.large nodes x (Haswell E5-2680 v3, 8 GB memory, 40GB Ultra Cloud Disk, 4x 135GB SSD Cloud Disk) Qian Wang, Rong Gu, Yihua Huang Nanjing University Reynold Xin Databricks Inc. Wei Wu, Jun Song, Junluan Xia Alibaba Group Inc.	2022, \$0.97 / TB Exoshuffle-CloudSort 100 TB for \$97 40 Amazon EC2 i4i.4xlarge nodes 1 Amazon EC2 r6i.2xlarge node Amazon S3 storage Frank Sifei Luan UC Berkeley Stephanie Wang UC Berkeley and Anyscale Samyukta Yagati, Sean Kim, Kenneth Lien, Isaac Ong, Tony Hong UC Berkeley SangBin Cho, Eric Liang Anyscale Ion Stoica UC Berkeley and Anyscale
Minute	2016, 37 TB Tencent Sort 512 nodes x (2 OpenPOWER 10-core POWER8 2.926 GHz, 512 GB memory, 4x Huawei ES3600P V3 1.2TB NVMe SSD, 100Gb Mellanox ConnectX4-EN) Jie Jiang, Lixiong Zheng, Junfeng Pu, Xiong Cheng, Chongqing Zhao Tencent Corporation Mark R. Nutter, Jeremy D. Schaub	2016, 55 TB Tencent Sort 512 nodes x (2 OpenPOWER 10-core POWER8 2.926 GHz, 512 GB memory, 4x Huawei ES3600P V3 1.2TB NVMe SSD, 100Gb Mellanox ConnectX4-EN) Jie Jiang, Lixiong Zheng, Junfeng Pu, Xiong Cheng, Chongqing Zhao Tencent Corporation Mark R. Nutter, Jeremy D. Schaub
Joule 10 ¹⁰ recs	2021, 138 KJoules RezSort 72 K records sorted / joule Intel i7-10700, 16GB RAM, Nsort, Ubuntu 18.04.5 LTS, 2 SK hynix Gold P31 1TB SSDs, 1 Samsung 980 Pro 2TB SSD Waleed Reda Université catholique de Louvain, KTH Royal Institute of Technology Dejan Kostić KTH Royal Institute of Technology	2022, 63 KJoules ELSAR 159 K records sorted / joule Intel Core i5-12600K, 32GB RAM, Ubuntu 20.04 Server, 4 WD_BLACK SN850 2TB Ani Kristo, Brown University Padmanabhan Pillai, Intel Labs Tim Kraska, MIT and Amazon



7.1 排序 (Sort) 的基本概念

◆排序：将一个按值任意的数据元素序列转换为一个按值有序的数据元素序列

对于含有 n 个记录的序列 $\{ R_1, R_2, \dots, R_n \}$, 对应的关键字序列为 $\{ k_1, k_2, \dots, k_n \}$, 确定一种置换关系

$$\sigma(1), \sigma(2), \dots, \sigma(n)$$

使得关键字序列满足：

$$k_{\sigma(1)} \leq k_{\sigma(2)} \leq \dots \leq k_{\sigma(n)}$$

或者

$$k_{\sigma(1)} \geq k_{\sigma(2)} \geq \dots \geq k_{\sigma(n)}$$

相应文件成为按关键字值有序的文件

$$\{ R_{\sigma(1)}, R_{\sigma(2)}, \dots, R_{\sigma(n)} \}$$

这一过程称为**排序**。



排序的功能和分类

◆排序的主要功能

- ✓能够将记录按关键字值任意排列的数据文件转换为一个记录按关键字值**有序排列**的数据文件
- ✓能够将一个按值任意排列的数据元素序列转换为一个按值有序排列的数据元素序列
- ✓能够**提高查找的时间效率**

◆排序的分类：按存储器的结构分

- ✓**内排序**：参加排序的数据量不大，以致于能够一次将参加排序的数据全部装入内存实现排序
- ✓**外排序**：当参加排序的数据量很大，以致于不能够一次将参加排序的数据全部装入内存，排序过程中需要不断地通过内存与外存之间的数据交换达到排序目的



排序性能

◆排序操作的性能评价主要包括:

- ✓时间性能
- ✓空间性能
- ✓稳定性

◆时间性能：排序过程中元素之间的比较次数与元素的移动次数

- ✓本章主要考虑最差情况的比较次数

◆空间性能：除了存放参加排序的元素之外，排序过程中所需要的其他辅助空间

◆稳定性：对于值相同的两个元素，排序前后的先后次序不变，则称该方法为稳定性排序方法，否则，称为非稳定性排序方法



排序中的约定

- ◆只针对一个数据元素（关键字）序列讨论排序方法
- ◆假设序列中具有 n 个数据元素（关键字），存放在数组元素中： $k[1], k[2], \dots, k[n]$ 中： $(k_1, k_2, k_3, \dots, k_{n-1}, k_n)$
- ◆排序结果按照数据元素（关键字）值的大小从小到大排列



—— 将具有 n 个数据元素（关键字）的序列转换为一个按照值的大小从小到大排列的序列通常要经过若干趟（Pass）



7.2 插入 (Insert) 排序

核 心 思 想

$k_{i,j}$

表示第*i*趟排序结束
时序列的第*j*个元素,
 $1 \leq i \leq n-1, 1 \leq j \leq n$

第*i*趟排序将序列的第*i*+1个元素**插入** 到一个大小为*i*、
且已经按值有序的子序列($k_{i-1,1}, k_{i-1,2}, \dots, k_{i-1,i}$) **合适位置**,
得到一个大小为*i*+1、且仍然按值有序的子序列($k_{i,1},$
 $k_{i,2}, \dots, k_{i,i+1}$)。

(1, 4, 6, 8, 12, 11, ...)



某趟排序过程

$\text{temp} \geq K[j]$
 $j=0$

49 38 97 76 65 13 27 50

$K[j+1]=K[j];$
 $j=j-1;$

... (若干趟后)

temp

65

38 49 65 76 97 13 27 50

$K[j+1]=\text{temp};$

38 49 65 76 97 13 27 50

一趟结束了



完整的排序过程

从第2个元素开始

初始:

49 38 97 76 65 13 27 50

第1趟:

38 49 97 76 65 13 27 50

第2趟:

38 49 97 76 65 13 27 50

第3趟:

38 49 76 97 65 13 27 50

第4趟:

38 49 65 76 97 13 27 50

第5趟:

13 38 49 65 76 97 27 50

第6趟:

13 27 38 49 65 76 97 50

第7趟:

13 27 38 49 50 65 76 97

结果

看一个完整的排序过程



插入排序算法实现

```
void insertSort(keytype k[], int n)
{
    int i, j;
    keytype temp;
    for (i = 1; i < n; i++)
    {
        temp = k[i];
        for (j = i - 1; j >= 0 && temp < k[j]; j--)
            k[j + 1] = k[j];
        k[j + 1] = temp;
    }
}
```

一趟排序

n-1趟排序



思考?

1. 排序的时间效率与什么直接有关?

答案

主要与排序过程中元素之间的比较次数直接有关

2. 若原始序列为一个按值递增的序列，则排序过程中一共要经过多少元素之间的比较?

答案

由于每一趟排序只需要经过一次元素之间的比较就可以找到被插入元素的合适位置，因此，整个 $n-1$ 趟排序一共要经过 $n-1$ 次元素之间的比较



思考? (续)

3. 若原始序列为一个按值递减的序列, 则排序过程中一共要经过多少次元素之间的比较?

答案

由于第*i*趟排序需要经过*i*次元素之间的比较才能找到被插入元素的合适位置, 因此, 整个*n-1*趟排序一共要经过

$$\sum_{i=1}^{n-1} i = n(n-1)/2$$

次元素之间的比较

若以最坏的情况考虑, 则插入排序算法的时间复杂度为 $O(n^2)$ 。

插入排序法是一种稳定性排序方法。



算法改进：采用折半查找确定插入位置

折半插入排序法

```
void insertBSort(keytype k[], int n){
    int i, j, low, high, mid;
    keytype temp;
    for (i = 1; i < n; i++) {
        temp = k[i];
        low = 0;
        high = i-1;
        while (low <= high){
            mid = (low + high) / 2;
            if (temp < k[mid])
                high = mid-1;
            else
                low = mid + 1;
        }
        for (j = i-1; j >= low; j--)
            k[j + 1] = k[j];
        k[low] = temp;
    }
}
```

采用折半查找
方法确定插入位置



练习1

请写一非递归算法，该算法在长度为 n 、且元素按值严格递增排列的顺序表 $A[1..n]$ 中采用 **折半查找法** 查找值不大于 k 的最大元素，若表中存在这样的元素，则算法返回该元素在表中的位置，否则返回0。

2 4 6 **8** 10 **12** 14 16 18 20

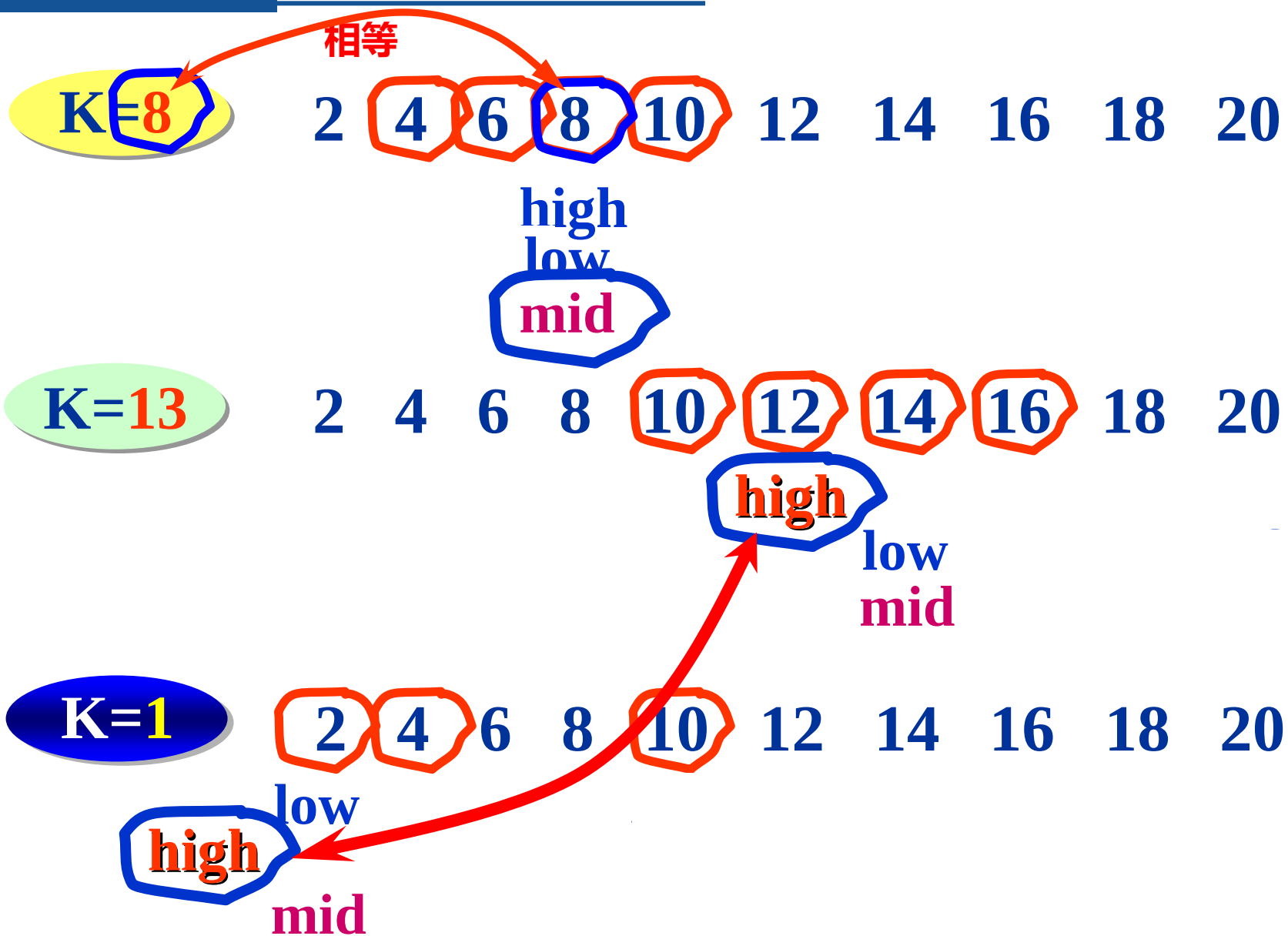
K=8

K=13

K=1



算法执行过程





算法实现

```
int searchB(keytype a[], int n, keytype k)
{
    int low = 0, high = n - 1, mid;
    while (low <= high)
    {
        mid = (low + high) / 2;
        if (a[mid] == k)
            return mid; //返回mid
        if (k > a[mid])
            low = mid + 1; //准备查找后半部分
        else
            high = mid - 1; //准备查找前半部分
    }
    return high; //返回high
}
```



插入排序算法分析

稳定性： 稳定

时间代价：

- 最佳情况： $n-1$ 次比较， 0 交换， $O(n)$
- 最差情况： 比较和交换次数为 $O(n^2)$
- 平均情况： $O(n^2)$

空间代价： $O(1)$



7.3 选择(Select)排序

核心思想

第 i 趟排序从序列的后 $n-i+1$ 个元素中**选择**一个值最小的元素，将其置于该 $n-i+1$ 个元素的最前面

排序的趟号	$\dots$	i	$n-i+1$	$\dots$
		1	n	
		2	$n-1$	
		3	$n-2$	
		$\dots$	$\dots$	

第 i 趟排序前，序列中未排序的元素个数

每一趟排序从序列中未排序的元素中**选择**一个值最小的元素，将其置于这些未排序的元素的最前面



每一趟排序过程

13 97 38 27 65 35 80 75

j

13 27 38 97 65 35 80 75

13 27 35 38 65 97 80 75

已排好序

j

不必交换



完整的排序过程

初始:	49	97	38	76	65	13	27	50
第1趟:	13	97	38	76	65	49	27	50
第2趟:	13	27	38	76	65	49	97	50
第3趟:	13	27	38	76	65	49	97	50
第4趟:	13	27	38	49	65	76	97	50
第5趟:	13	27	38	49	50	76	97	65
第6趟:	13	27	38	49	50	65	97	76
第7趟:	13	27	38	49	50	65	76	97

结果

$n-1$ 趟

看一个完整的过程



算法实现

```
void selectSort(keytype k[], int n){  
    int i, j, d;  
    keytype temp;  
    for (i = 0; i < n - 1; i++)  
    {  
        d = i;  
        for (j = i + 1; j < n; j++)  
            if (k[j] < k[d])  
                d = j;  
        if (d != i)  
        {  
            //最小值元素非未排序元素的第一个元素时  
            temp = k[d];  
            k[d] = k[i];  
            k[i] = temp;  
        }  
    }  
}
```

寻找值最小的元素,并记录其位置

n-1趟排序



思考？

若原始序列为一个按值**递增**的序列，则排序过程中一共要经过多少元素之间的比较？若原始序列为一个按值**递减**的序列，则排序过程中要经过多少元素之间的比较？

答案：

无论原始序列为什么状态，第*i*趟排序都需要经过*n-i*次元素之间的比较，因此，整个排序过程中元素之间的比较次数为 $\sum_{i=1}^{n-1} (n-i) = n(n-1)/2$ 。

结论 选择排序元素之间的比较次数与原始序列中元素的分布状态**无关**

时间复杂度为 $O(n^2)$ 。

不稳定
排序方法



选择排序算法分析

稳定性:	不稳定
时间代价:	➤ 比较次数: $O(n^2)$ ➤ 交换次数: $n-1$ ➤ 总时间代价: $O(n^2)$
空间代价:	$O(1)$



7.4 冒泡 (bubble) 排序

核心思想

特点

值大的元素往后 “沉”
值小的元素向前 “浮”

第 i 趟排序对序列的前 $n-i+1$ 个元素从第一个元素开始依次作如下操作:相邻的两个元素比较大小,若前者大于后者,则两个元素交换位置,否则不交换位置

效果

该 $n-i+1$ 个元素中最大值元素移到该 $n-i+1$ 个元素的最后



完整的排序过程

$n=8$

$n-i+1$

初始 49 97 38 13 27 50 76 65

第1趟 49 38 13 27 50 76 65 97

第2趟 38 13 27 49 50 65 76 97

第3趟 13 27 38 49 50 65 76 97

第4趟 13 27 38 49 50 65 76 97

排序结束



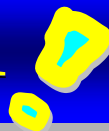
结果

13 27 38 49 50 65 76 97

j

$j+1$

排序总趟数可以小于 $n-1$





算法实现

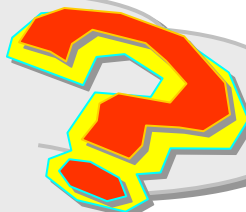
```
void bubbleSort(keytype k[], int n)
{
    int i, j, flag = 1;
    keytype temp;
    for (i = n - 1; i > 0 && flag == 1; i--)
    {
        flag = 0; //每趟排序前标志flag置0
        for (j = 0; j < i; j++)
            if (k[j] > k[j + 1])
            {
                temp = k[j];
                k[j] = k[j + 1];
                k[j + 1] = temp; //交换两个元素的位置
                flag = 1;        //标志flag置1
            }
    }
}
```



算法性能分析

泡排序法的排序趟数与原始序列中数据元素的排列有关，因此，排序的趟数为一个范围，即 $[1..n-1]$

什么情况下只要排序一趟
什么情况下要排序 $n-1$ 趟



$O(n^2)$

泡排序方法比较适合于
参加排序的序列的原始状态
基本有序的情况



泡排序法是**稳定性排序方法**。

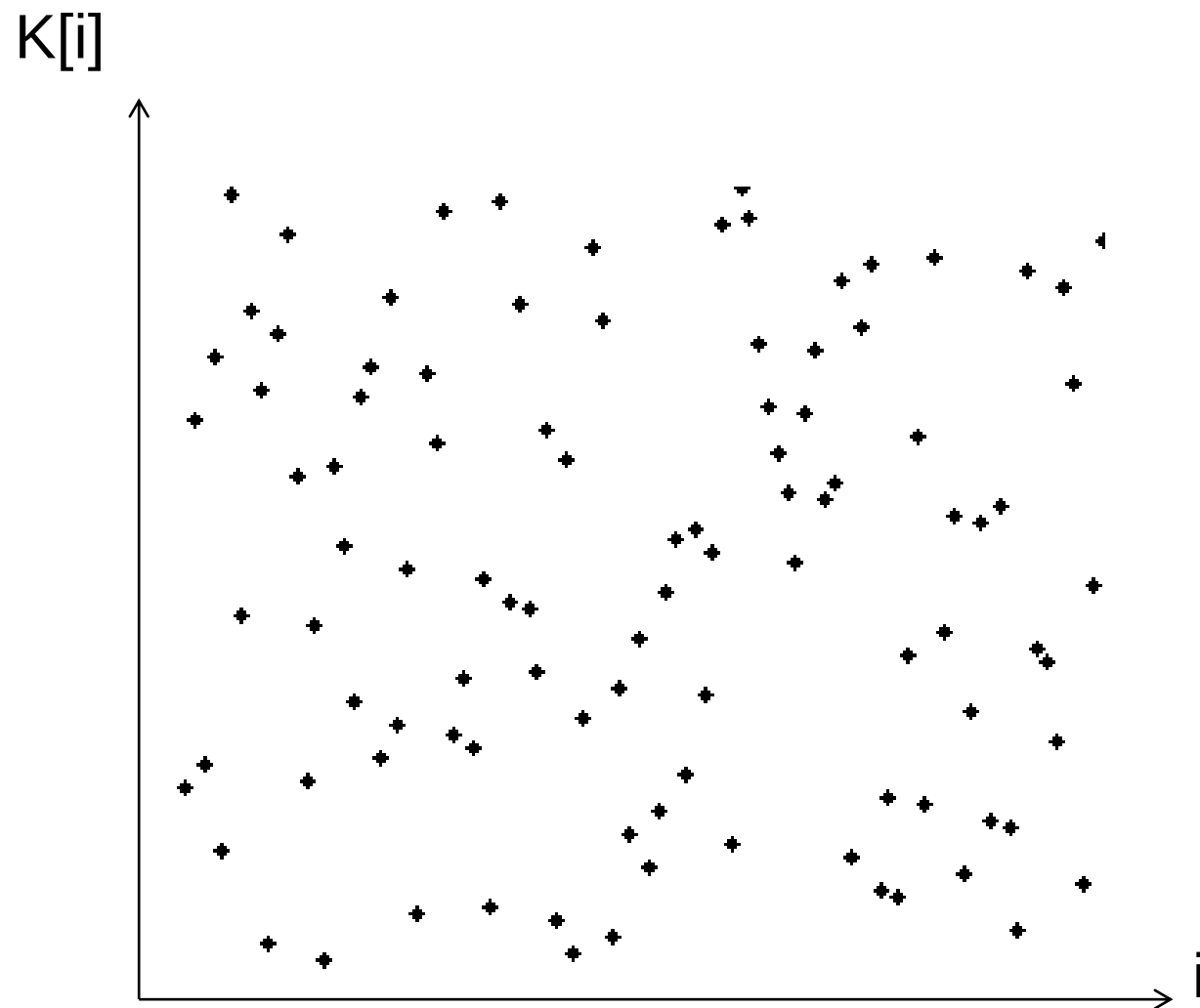


(改进) 冒泡排序算法性能分析

稳定性:	稳定
时间代价:	➤ 时间代价: ◆ 最小时间代价为$O(n)$, 最佳情况下只运行第一轮循环 ◆ 一般情况下为$O(n^2)$
空间代价:	$O(1)$

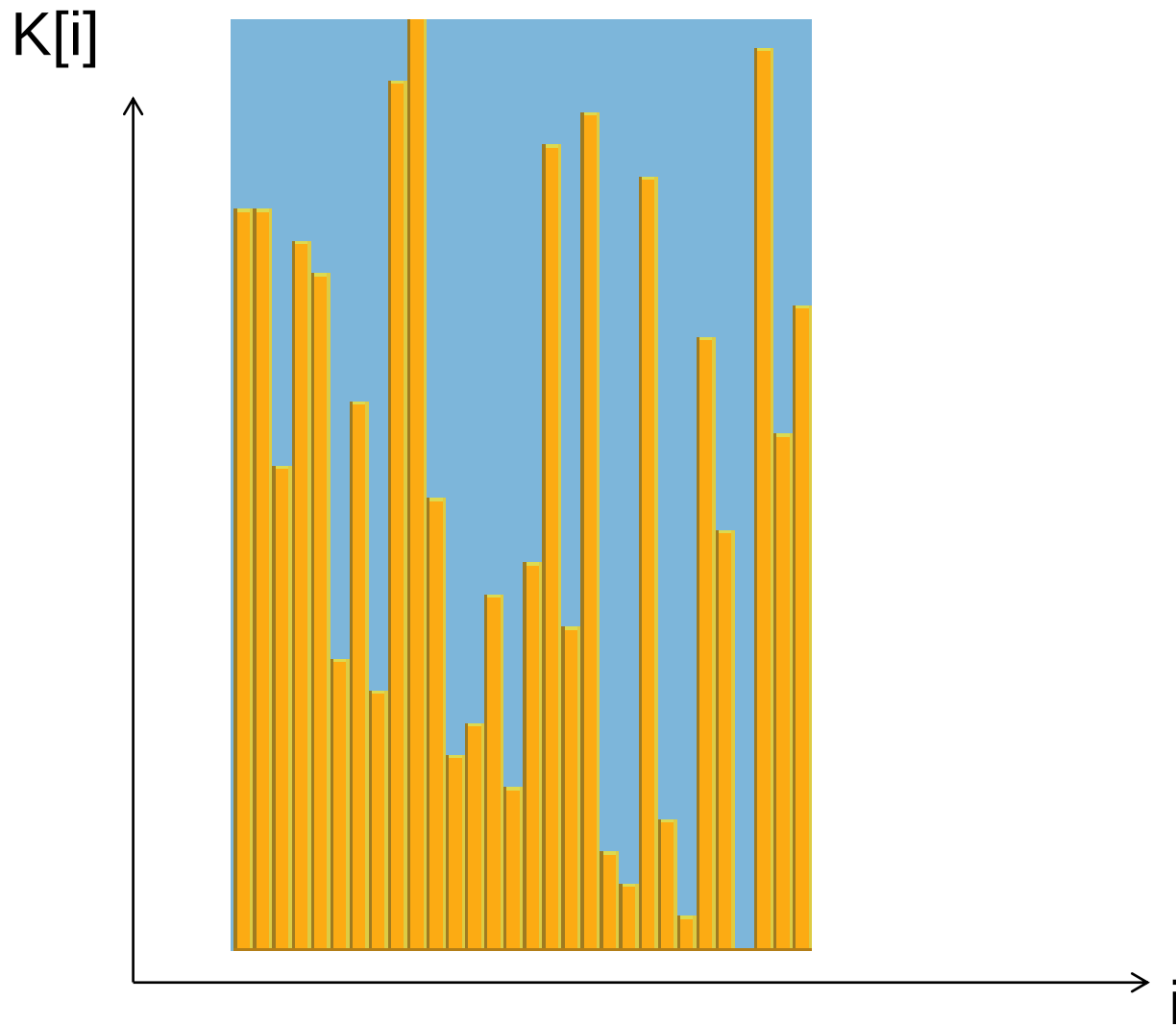


思考：什么算法





思考：什么算法





7.5 谢尔(Shell)排序

由Donald Shell提出

核心思想

缩小增量排序法

首先确定一个元素的间隔数gap。

将参加排序的元素按照gap分隔成若干个子序列(即分别把那些位置相隔为gap的元素看作一个子序列),然后对各个子序列采用**某一种排序方法**进行排序;此后减小gap值,重复上述过程,直到 $gap < 1$

一种减小gap的方法:

$$\begin{aligned} gap_1 &= \lfloor n/2 \rfloor \\ gap_i &= \lfloor gap_{i-1}/2 \rfloor \quad i = 2, 3, \dots \end{aligned}$$



排序过程

初 始 49 97 38 50 76 65 13 27 25

第1趟
gap=4 25 65 13 27 49 97 38 50 76

第2趟
gap=2 13 27 25 50 38 65 49 97 76

第3趟
gap=1 [13 25 27 38 49 50 65 76 97]

结果

gap=1

13 25 27 38 49 50 65 76 97

子序列内采用
泡排序方法



算法实现

```
void shellSort(keytype k[], int n){
    int i, j, flag, gap = n;
    keytype temp;
    while (gap > 1) {
        gap = gap / 2;
        do {
            flag = 0; //每趟排序前,标志flag置0
            for (i = 0; i < n-gap; i++){
                j = i + gap;
                if (k[i] > k[j]) {
                    temp = k[i];
                    k[i] = k[j];
                    k[j] = temp;
                    flag = 1;
                }
            }
        } while (flag != 0);
    }
}
```

```
void shellSort(int v[], int n)//from K & R
{
    int gap, i, j, temp;
    for (gap = n / 2; gap > 0; gap /= 2)
        for (i = gap; i < n; i++)
            for (j = i - gap;
                j >= 0 && v[j] > v[j + gap];
                j -= gap)
            {
                temp = v[j];
                v[j] = v[j + gap];
                v[j + gap] = temp;
            }
}
```

from K & R



算法实现 (续)

```
void shellSort(keytype k[], int n)
{
    int i, j, gap = n;
    keytype temp;
    while (gap > 1) {
        gap = gap / 2;
        // 使用插入排序实现子序列排序
        for (i = gap; i < n; i++){
            temp = k[i];
            for (j = i; j >= gap && k[j - gap] > temp; j -= gap)
                k[j] = k[j - gap];
            k[j] = temp;
        }
    }
}
```

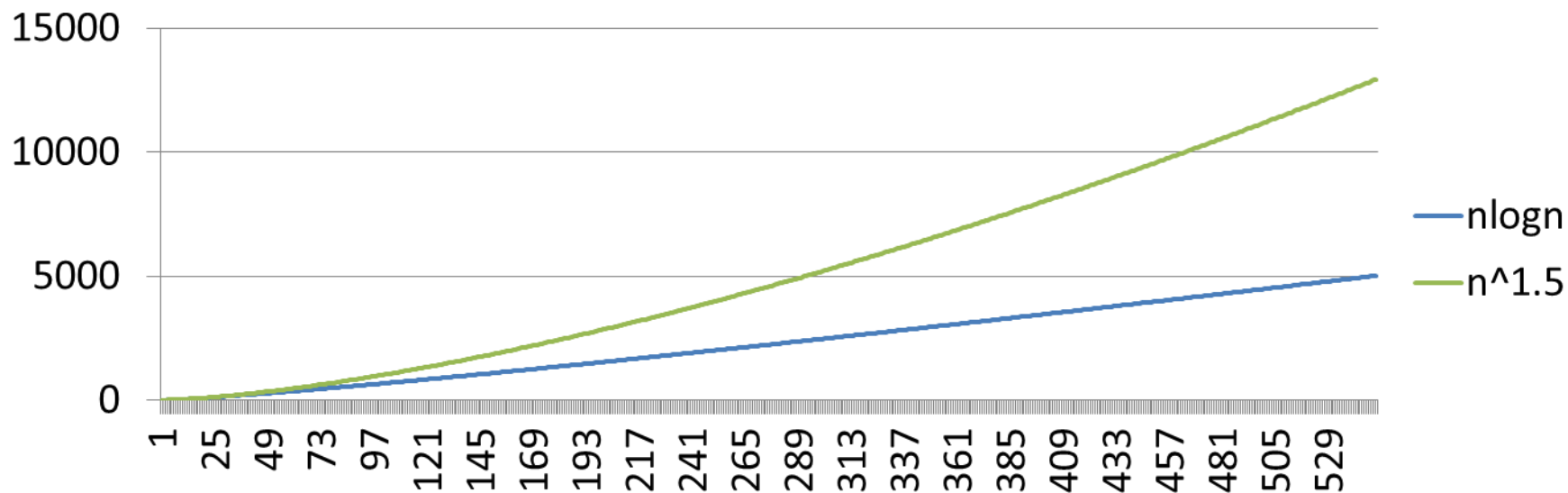
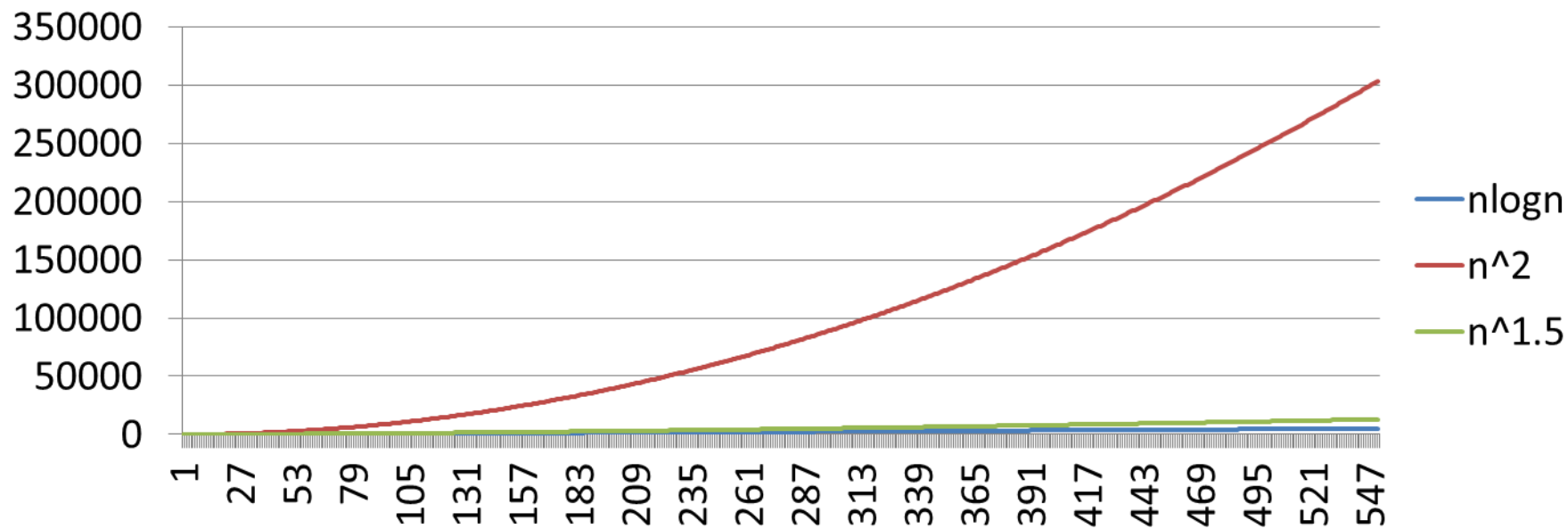


Shell排序算法分析

稳定性:	不稳定
时间代价:	$O(n\log_2 n)$ 与 $O(n^2)$ 之间 (通常 $<O(n^{3/2})$)
空间代价:	$O(1)$



不同时间复杂度的性能分析





7.6 堆 (Heap) 排序

一.堆的定义

1. n 个元素的序列 $(k_1, k_2, \dots, k_n)$, 当且仅当满足

$$(1) \begin{cases} k_i \geq k_{2i} \\ k_i \geq k_{2i+1} \end{cases} \quad \text{或者} \quad (2) \begin{cases} k_i \leq k_{2i} \\ k_i \leq k_{2i+1} \end{cases} \quad i=1, 2, 3, \dots, \lfloor n/2 \rfloor$$

称该序列为一个**堆积**(heap), 简称**堆**。

称满足条件(1)的堆为**大顶堆**, 称满足条件(2)的堆为**小顶堆**。



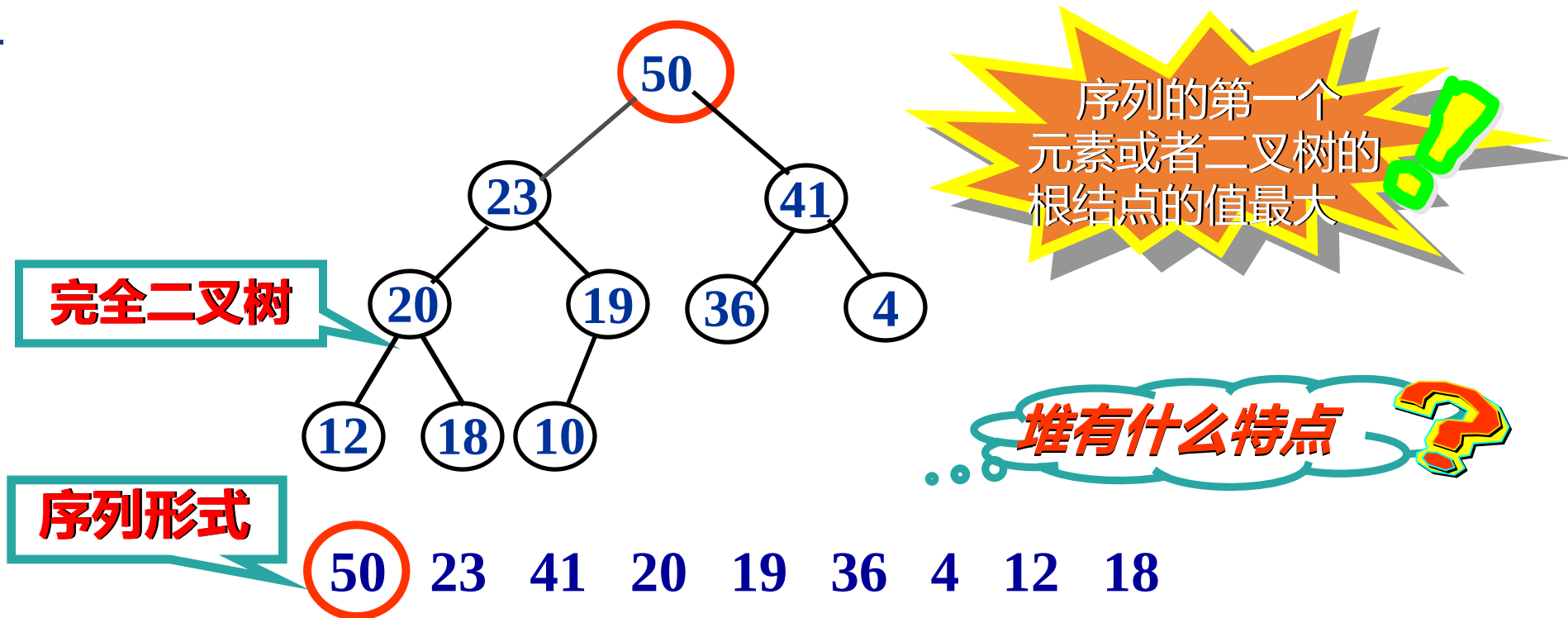
50 23 41 20 19 36 4 12 18

一个(大顶)堆



堆的性质

- ◆堆是一棵完全二叉树，二叉树中任何一个分支结点的值都大于或者等于它的孩子结点的值，并且每一棵子树也满足堆的特性





堆排序

二.排序的核心思想

第 i 趟排序将序列的前 $n-i+1$ 个元素组成的子序列转换为一个堆积,然后将堆的第一个元素与堆的最后那个元素交换位置

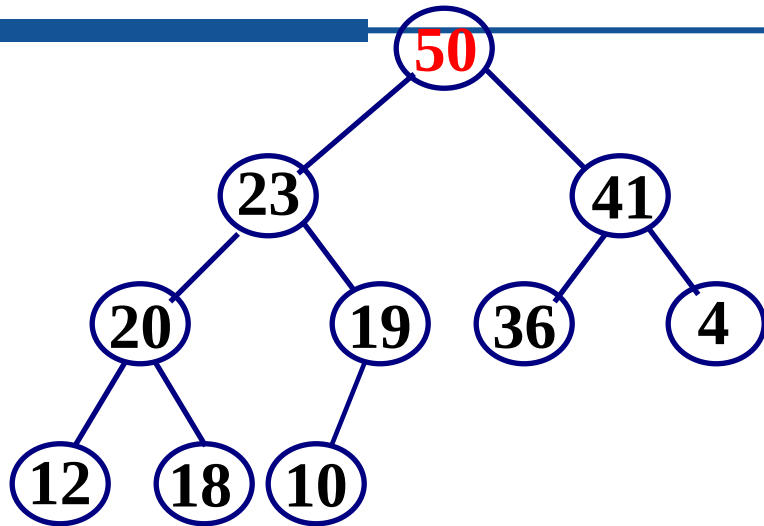
三.排序步骤

建初始堆积

- ①. 将原始序列转换为第一个堆。
- ②. 将堆的第一个元素与堆积的最后那个元素交换位置。
(即“去掉”最大值元素)
- ③. 将“去掉”最大值元素后剩下的元素组成的子序列重新转换一个新的堆。
- ④. 重复上述过程的第2至第3步 $n-1$ 次。



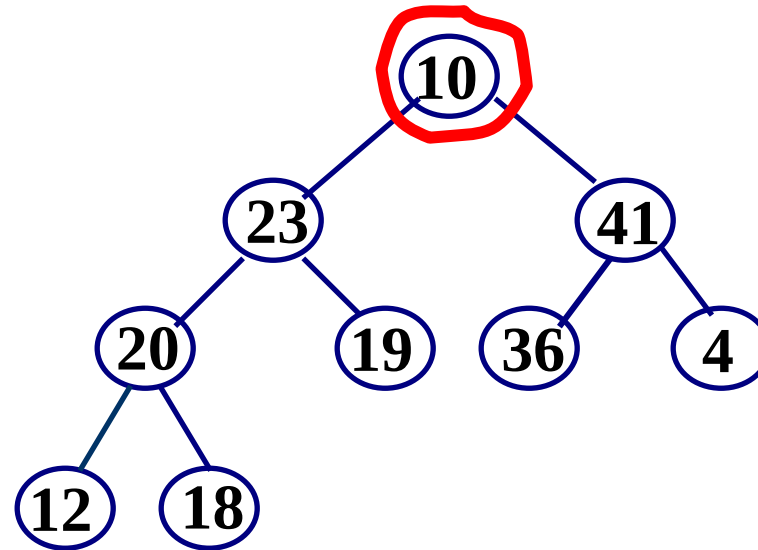
堆排序过程



...

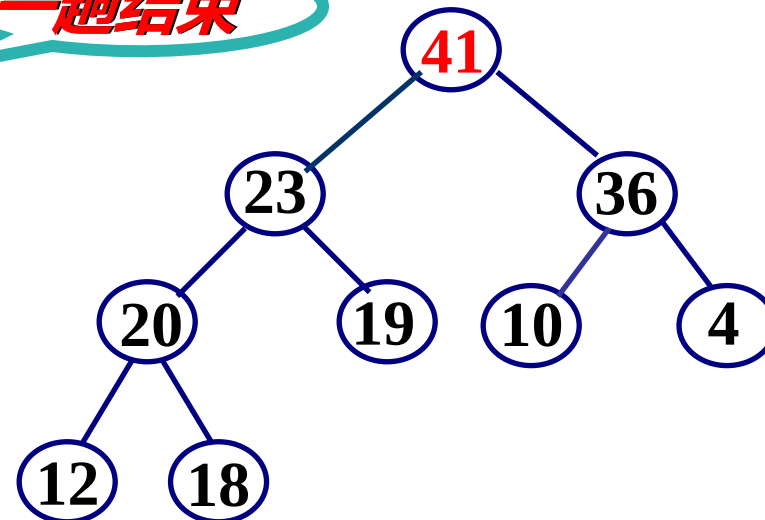
50	23	41	20	19	36	4	12	18	10
10	23	41	20	19	36	4	12	18	50
41	23	36	20	19	10	4	12	18	50
18	23	36	20	19	10	4	12	41	50
36	23	18	20	19	10	4	12	41	50
12	23	18	20	19	10	4	36	41	50

...



将根结点往叶结点方向移

一趟结束





四. 调整子算法

```
void adjust(keytype k[], int i, int n){
    int j;
    keytype temp;
    temp = k[i];
    j = 2 * i + 1;
    while (j < n) {
        if (j + 1 < n && k[j] < k[j + 1])
            j++;
        if (temp < k[j]) {
            k[(j - 1) / 2] = k[j];
            j = 2 * j + 1;
        }
        else
            break;
    }
    k[(j - 1) / 2] = temp;
}
```

功能

向下调整结点*i*的位置，使得其祖先结点值都比其大。如果一棵树仅根结点*i*不满足堆条件，通过该函数可将其调整为一个堆

K:序列

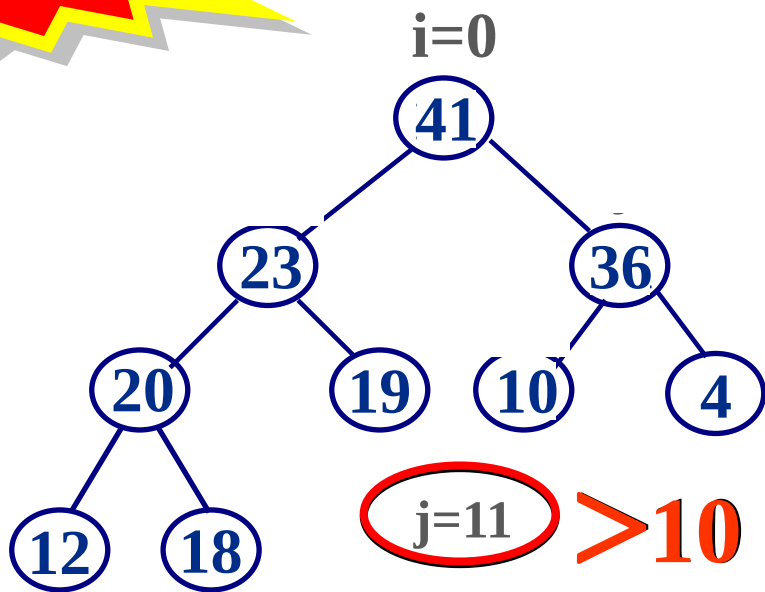
i: 被调整的二叉树的根的序号

n: 被调整的二叉树的结点数



调整过程

一个新的堆积



temp

10

$n=10$

```
void adjust(keytype k[], int i, int n){
    int j;
    keytype temp;
    temp = k[i];
    j = 2 * i + 1;
    while (j < n) {
        if (j + 1 < n && k[j] < k[j + 1])
            j++;
        if (temp < k[j]) {
            k[(j - 1) / 2] = k[j];
            j = 2 * j + 1;
        }
        else
            break;
    }
    k[(j - 1) / 2] = temp;
}
```

功能 向下调整结点*i*的位置，使得其祖先结点值都比其大。如果一棵树仅根节点不满足堆条件，通过该函数可将其调整为一个堆。

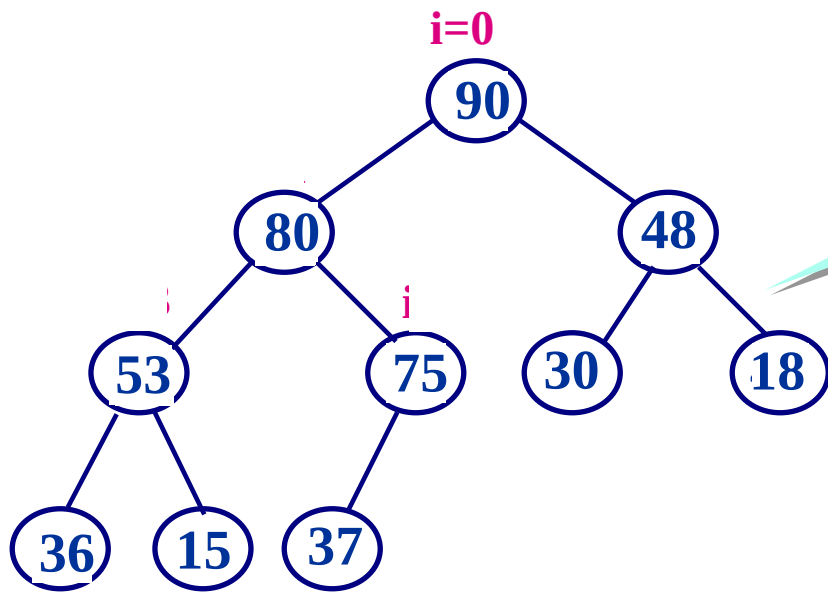


五. 建立初始堆

从二叉树的最后那个分支结点(编号为 $i = \lfloor n/2 - 1 \rfloor$)开始,依次将编号为 i 的结点为根的二叉树**转换**为一个堆, 每转换一棵子树, 做一次 i 减1, 重复上述过程, 直到将 $i=0$ 的结点为根的二叉树转换为堆

75 36 18 53 80 30 48 90 15 37

90 80 48 53 75 30 18 36 15 37



初始堆积



完整的堆排序过程

原始 75 36 18 53 80 30 48 90 15 37

第1趟 37 80 48 53 75 30 18 36 15 90

第2趟 15 75 48 53 37 30 18 36 80 90

第3趟 15 53 48 36 37 30 18 75 80 90

第4趟 18 37 48 36 15 30 53 75 80 90

第5趟 18 37 30 36 15 48 53 75 80 90

第6趟 15 36 30 18 37 48 53 75 80 90

第7趟 15 18 30 36 37 48 53 75 80 90

第8趟 15 18 30 36 37 48 53 75 80 90

第9趟 15 18 30 36 37 48 53 75 80 90

初始堆积



六. 堆排序算法

```
void heapSort(keytype k[], int n)
{
    int i,
    keytype temp;
    for (i = n / 2 - 1; i >= 0; i--)
        adjust(k, i, n);
    for (i = n - 1; i >= 1; i--)
    {
        temp = k[i];
        k[i] = k[0];
        k[0] = temp;
        adjust(k, 0, i);
    }
}
```

建初始堆积

具体排序

n-1趟排序

$O(n \log_2 n)$



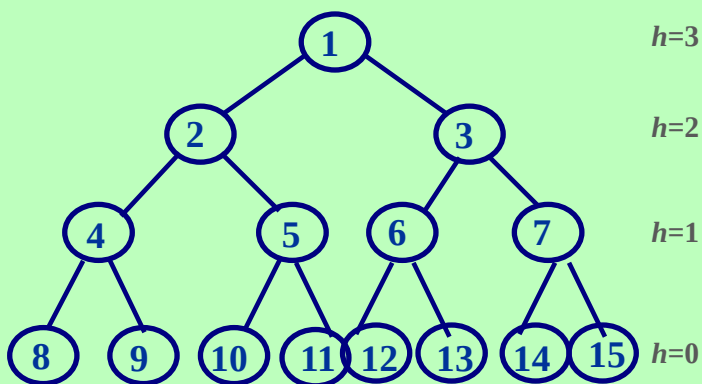
时间复杂度分析

```
for(i=n/2-1;i>=0;i--)  
    adjust(K,i,n);
```

建初始堆积复杂度

第 h 层节点数 $\leq \left\lceil 2^{(\lg n - h) - 1} \right\rceil = \left\lceil \frac{2^{\lg n}}{2^{h+1}} \right\rceil = \left\lceil \frac{n}{2^{h+1}} \right\rceil$

时间复杂度 = $\sum_{h=0}^{\lceil \lg n \rceil} \frac{n}{2^{h+1}} O(h) = O\left(n \sum_{h=0}^{\lceil \lg n \rceil} \frac{h}{2^{h+1}}\right)$



$$\leq O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h}\right) = O(n)$$



堆排序算法分析

稳定性:	不稳定
时间代价:	$O(n\log_2 n)$
空间代价:	$O(1)$



7.7 二路归并 (Merge) 排序

一.什么是二路归并?

将两个位置相邻、并且各自按值有序的子序列合并为一个按值有序的子序列的过程称为二路归并

$$\underbrace{(K_s, K_{s+1}, K_{s+2}, \dots, K_u)}_{\text{有序子序列}} \underbrace{(K_{u+1}, K_{u+2}, K_{u+3}, \dots, K_v)}_{\text{有序子序列}}$$
$$(X_s, X_{s+1}, X_{s+2}, X_{s+3}, \dots, X_v)$$

其中

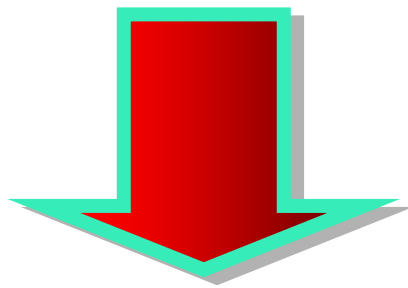
$$K_s \leq K_{s+1} \leq K_{s+2} \leq \dots \leq K_u$$
$$K_{u+1} \leq K_{u+2} \leq K_{u+3} \leq \dots \leq K_v$$
$$X_s \leq X_{s+1} \leq X_{s+2} \leq X_{s+3} \leq \dots \leq X_v$$



例：归并过程

$\dots K_s, K_{s+1}, K_{s+2}, K_{s+3}, K_{s+4}, K_{s+5} \quad K_{s+6}, K_{s+7}, K_{s+8}, K_{s+9} \dots$

(12, 45, 78, 90, 100, 125) (27, 32, 51, 93)

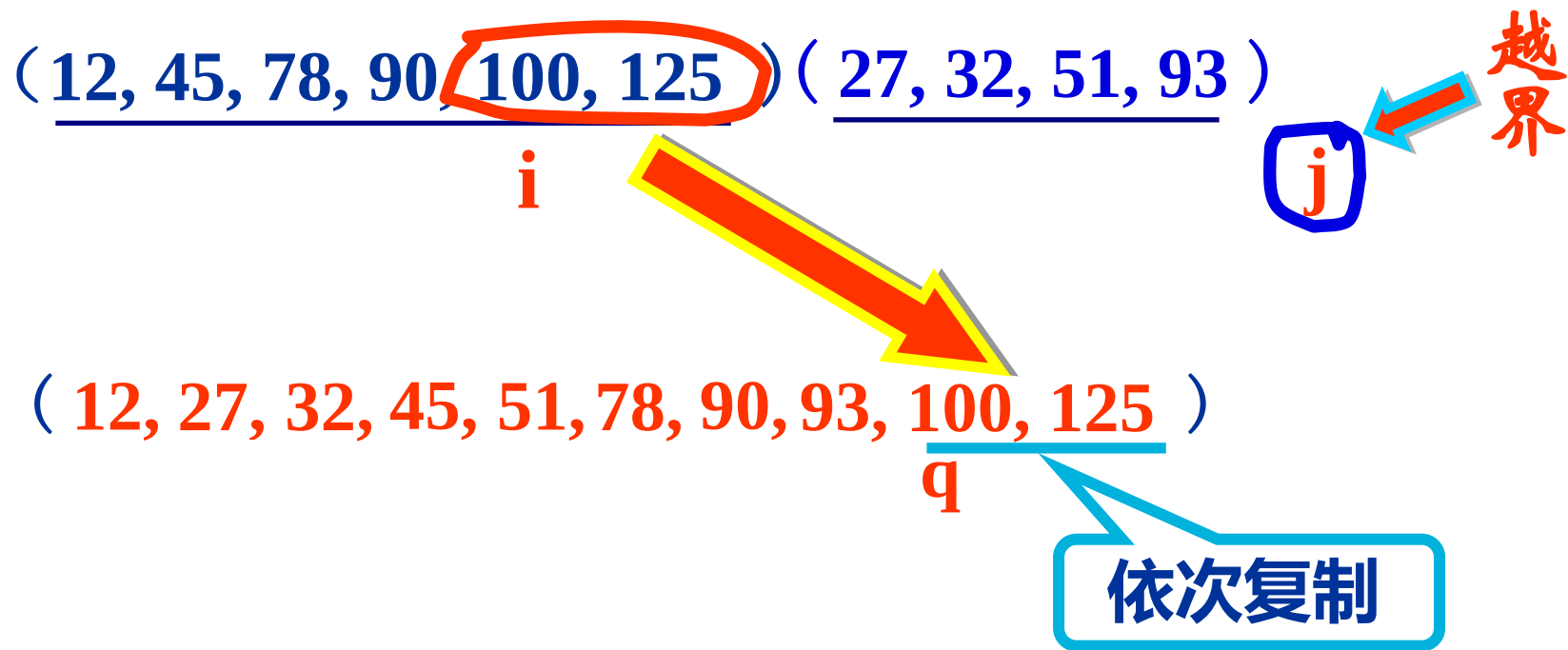


(12, 27, 32, 45, 51, 78, 90, 93, 100, 125)

二路归并



例：归并过程（续）





合并算法实现

```
void merge(keytype x[], keytype tmp[],
           int left, int leftend, int rightend)
{
    int i = left, j = leftend + 1, q = left;
    while (i <= leftend && j <= rightend)
        if (x[i] <= x[j])
            tmp[q++] = x[i++];
        else
            tmp[q++] = x[j++];
    while (i <= leftend)
        tmp[q++] = x[i++];
    while (j <= rightend)
        tmp[q++] = x[j++];
    for (i = left; i <= rightend; i++)
        x[i] = tmp[i];
}
```

功能 将两个位置
相邻且按值有序子序列
合并为一个按值有
序的序列

复制第一个子
序列的剩余部分

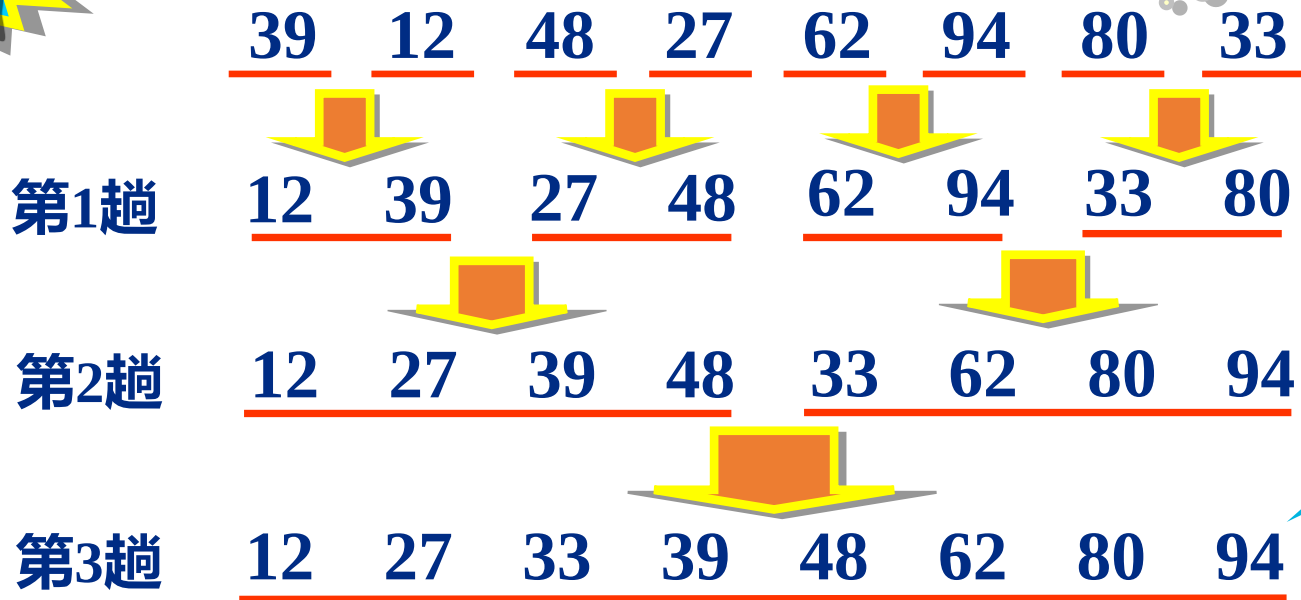
复制第二个子
序列的剩余部分

将合并后内容复
制回原数组



二. 核心思想

第 i 趟排序将序列的 $\lfloor \frac{n}{2^{i-1}} \rfloor$ 个长度为 2^{i-1} 的按值有序的子序列依次两两合并为 $\lfloor \frac{n}{2^i} \rfloor$ 个长度为 2^i 的按值有序的子序列



分别看成8个大小为1的有序子序列

结果



三. 归并排序算法

```
void mergeSort(keytype k[], int n){
    keytype *tmp;
    tmp = (keytype *)malloc(sizeof(keytype) * n);
    if (tmp != NULL) {
        mSort(k, tmp, 0, n - 1);
        free(tmp);
    }
    else printf("No space for tmp array!!!\n");
}

void mSort(keytype k[], keytype tmp[], int left, int right){
    int center;
    if (left < right) {
        center = (left + right) / 2;
        mSort(k, tmp, left, center);
        mSort(k, tmp, center + 1, right);
        merge(k, tmp, left, center, right);
    }
}
```



归并排序算法分析

稳定性:	稳定
时间代价:	$O(n\log_2 n)$ (不依赖于原始数据的输入情况, 最大、最小及平均时间代价均为 $O(n\log_2 n)$)
空间代价:	$O(n)$



7.8 快速 (Quick) 排序

一、核心思想

可以选第一个或者最后一个、或位置居中的那个元素作为分界元素

划分元素、基准元素、枢轴、轴值、支点

从当前参加排序的元素中任选一个元素(通常称之为分界元素 **pivot**)与当前参加排序的那些元素进行比较,凡是小于分界元素的元素都移到分界元素的前面,凡是大于分界元的元素都移到分界元素的后面,分界元素将当前参加排序的元素分成前后两部分,而分界元素处在排序的最终位置。然后,分别对这两部分中大小大于1的部分重复上述过程,直到排序结束

递归过程

快速排序算法最早由图灵奖获得者**Tony Hoare**设计出来,被列为20世纪十大算法之一。也是目前实践中已知最快的排序算法



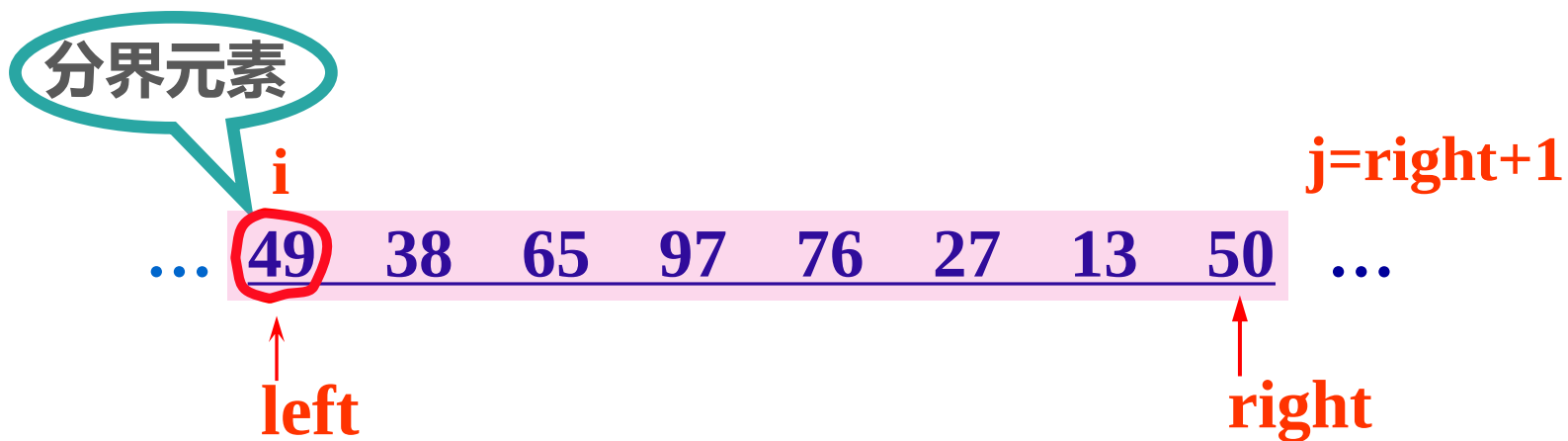
二. 算法步骤

算法中用到的变量

left: 当前参加排序的那些元素的第一个元素在序列中的位置, 初始值为0。

right: 当前参加排序的那些元素的最后那个元素在序列中的位置, 初始值为 $n-1$ 。

i, j: 两个位置变量, 初始值分别为left 与 $\text{right}+1$ 。





算法步骤 (续)

分界元素 $K[s]$

1. 反复执行动作 $i+1 \rightarrow i$, 直到 $K[\text{left}] \leq K[i]$ 或者 $i = \text{right}$ 。
反复执行动作 $j-1 \rightarrow j$, 直到 $K[\text{left}] \geq K[j]$ 或者 $j = \text{left}$ 。
2. 若 $i < j$, 则 $K[i]$ 与 $K[j]$ 交换位置, 转到第1步。
3. 若 $i \geq j$, 则 $K[\text{left}]$ 与 $K[j]$ 交换位置, 到此, 分界元素 $K[\text{left}]$ 的最终位置已经确定, 然后对被 $K[\text{left}]$ 分成的两部分中个数大于1 的部分重复上述过程, 直到排序结束。

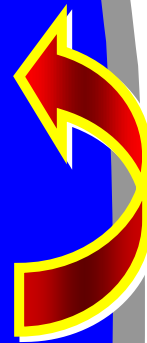
j



示例

原始序列

				s	t		
13	27	38	49	50	65	76	97
49	38	65	97	76	27	13	50



步骤

1. 反复执行动作 $i+1 \rightarrow i$, 直到 $K[\text{left}] \leq K[i]$ 或者 $i = \text{right}$ 。
反复执行动作 $j-1 \rightarrow j$, 直到 $K[\text{left}] \geq K[j]$ 或者 $j = \text{left}$ 。
2. 若 $i < j$, 则 $K[i]$ 与 $K[j]$ 交换位置, 转到第1步。
3. 若 $i \geq j$, 则 $K[\text{left}]$ 与 $K[j]$ 交换位置, 到此, 分界元素 $K[\text{left}]$ 的最终位置已经确定, 然后对被 $K[\text{left}]$ 分成的两部分中大小大于1的部分重复上述过程, 直到排序结束。



算法实现（本例中找中间的元素作为分界元素）

```
void qsort(keytype v[], int left, int right)
{
    int i, last;

    if (left >= right)
        return;
    swap(v, left, (left + right) / 2); //move partition elem to v[0]
    last = left;
    for (i = left + 1; i <= right; i++) //partition
        if (v[i] < v[left])
            swap(v, ++last, i);
    swap(v, left, last); //restore partition elem
    qsort(v, left, last);
    qsort(v, last + 1, right);
}
```

```
void swap(keytype v[], int i, int j)
{
    keytype tmp;
    tmp = v[i];
    v[i] = v[j];
    v[j] = tmp;
}
```

//主算法

```
void quickSort(keytype k[], int n)
{
    qsort(k, 0, n - 1);
}
```



快速排序算法分析

稳定性	不稳定
最差情况:	◆ 时间代价: $O(n^2)$ ◆ 空间代价: $O(n)$
最佳情况:	◆ 时间代价: $O(n\log_2 n)$ ◆ 空间代价: $O(\log_2 n)$
平均情况:	◆ 时间代价: $O(n\log_2 n)$ ◆ 空间代价: $O(\log_2 n)$



使用库函数实现快速排序

◆快速排序库函数qsort (stdlib.h头文件)

```
//<stdlib.h>中提供的快速排序函数
//qsort: 负责框架调用和给(*comp)传递所需参数
//根据(*comp)的返回值决定如何交换数组的成员;
//base: 需要排序的数组首地址 (void*指向任意类型的数组);
//num: 参与排序的数组元素的个数;
//wid: 数组中每个元素所占用的字节数;
//comp: 指向数组元素比较函数的指针, 接收两个元素的地址
//(*comp): 函数负责比较两个元素, 返回负数、正数和0,
//分别表示第一个参数先于、后于和等于第二个参数
void qsort( void *base, size_t num, size_t wid,
            int (*comp)(const void *e1, const void *e2) );
```



示例：使用qsort排序

◆给定一个具有n个元素的double型数组，使用qsort()对数组元素按升序和降序排序

```
double a[100]; //需要排序的数组
int n; //数组中实际元素的个数
...
//按升序排序
qsort(a, n, sizeof(double), rise_double);
// 按降序排序
qsort(a, n, sizeof(double), fall_double);
```

```
//升序排序，最基础的写法
int rise_double(const void *p1, const void *p2)
{
    double *d1=(double *)p1; //转换为目标类型
    double *d2=(double *)p2; //转换为目标类型
    if(*d1 < *d2) return -1; //根据大小返回值
    else if(*d1 > *d2) return 1;
    else return 0;
}
```

```
//降序排序，直接类型转换
int fall_double(const void *p1, const void *p2)
{
    if ( *(double *)p1 > *(double *)p2 ) return -1;
    if ( *(double *)p1 < *(double *)p2 ) return 1;
    if ( *(double *)p1 == *(double *)p2 ) return 0;
}
```



七种排序算法比较

排序算法	平均情况	最好情况	最坏情况	辅助空间	稳定性
插入排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	稳定
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
冒泡排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	稳定
希尔排序	$O(n \log n) \sim O(n^2)$	$O(n^{1.3})$	$O(n^2)$	$O(1)$	不稳定
堆排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	不稳定
二路归并排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	稳定
快速排序	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n) \sim O(n)$	不稳定

从算法性质来看：

- ◆ 简单算法：冒泡、选择、插入
- ◆ 改进算法：谢尔、堆、归并、快速

从时间复杂度来看：

- ◆ 平均情况：后3种改进算法 > 谢尔 >> 简单算法
- ◆ 最好情况：冒泡和插入排序要更好一些
- ◆ 最坏情况：堆和归并排序要好于快速排序及简单排序

从空间复杂度来看：

- ◆ 归并排序有额外空间要求，快速排序也有相应空间要求，堆排序则基本没有

从稳定性来看：

- ◆ 除了简单排序，归并排序不仅速度快，而且还稳定



其他排序算法：桶(Bucket)排序法* (计数排序)

一、核心思想

假设 $a_1, a_2 \dots a_n$ 由小于 M 的正整数组成，桶排序的基本原理是使用一个大小为 M 的数组 C (初始化为0，称为桶bucket)，当处理 a_i 时，使 $C[a_i]$ 增1。最后遍历数组 C 输出排序后的表

基本思想是由E.J.Issac R.C.Singleton提出来。桶排序并不是比较排序，不受到 $O(n \log n)$ 下限的影响，是最简单、最快的排序，时间复杂度为 $O(M+N)$



小结：查找

◆排序的基本概念

- ✓排序

- ✓性能分析：稳定性、时间复杂度、空间复杂度

◆简单排序算法

- ✓插入排序

- ✓选择排序

- ✓冒泡排序

◆改进排序算法

- ✓谢尔排序

- ✓堆排序

- ✓二路归并排序

- ✓快速排序